

该技术针对检测产生的误报进行识别。

源码静态分析背景

大多数的软件缺陷都与源码相关



源码静态分析是软件缺陷检测的常用方法之一

方法名称	方法说明	特点	典型工具
人工（黑盒）			
人工（白盒）			
模糊测试			
源码静态分析	使用符号执行、程序依赖分析、抽象解释等方式检测缺陷及漏洞	白盒、检测速度快、存在误报	Coverity、CheckMarx、Fortify
软件成分分析 （二进制代码）			

源码静态分析的作用

- 开发过程中就能自动检测代码缺陷问题：黑盒测试一般都是在集成阶段进行的系统测试，而源代码检测除了在集成测试阶段可进行测试外，还可在开发过程中进行实时监测，解决代码问题。甚至在需求和设计阶段就可提出代码层面的可验证的安全需求和约束。将软件的安全质量问题发现和解决窗口提前，降低研发成本，缩短研发周期，提高产品质量。
- 检测环境方便，测试覆盖率高：无需搭建复杂的系统运行环境，只需程序员保证源代码可编译即可检测，全量代码检测能够实现较高的代码覆盖率。
- 全面检测代码安全性、可靠性、性能、以及代码规范性问题：开发人员主要聚焦于业务逻辑实现，源码静态分析聚焦于源代码本身的安全、可靠、性能和规范性，除了业界通用标准规则外，企业内部公共组建的使用规范还可定制规则进行检测。
- 持续提升开发测试人员的编码水平：在代码自动检测、人工审计和问题修复过程中，能持续提升开发和测试人员代码水平，以便编写出更加安全、更加可靠、更加规范的程序。
- 源码检测是软件设计的必然要求：源码检测和验证是软件设计的必然需求，将源代码编译成软件产品的构建过程已经完全自动化。因此，编写源代码就是软件设计的重要过程，对源代码的测试验证就是对软件设计的自动评审。

源码静态分析技术概览



源码静态分析技术分类

按照技术分类：

- 基于模式匹配
- 基于数据流分析
- 基于符号执行
- 基于抽象解释
- 基于演绎推理
- 基于模型检测

按照使用方式分类：

- 非编译型（源码）：无需编译即可检测，对检测环境无要求，如 PMD
- 编译检测一体型（字节码）：源码需要编译才可检测，并且是编译的同时就进行检测，如：klocwork、cpptest
- 编译捕获和检测分离型（字节码）：先编译通过才能检测，如：fortify, spotbugs, 该类型的优点是：用户可以在开发环境中进行编译捕获，然后将中间文件拿到检测环境中进行检测，检测服务器无需搭建编译环境

Ps：基于二进制码进行检测的工具，如：angr, mcsema, bap, IDA Home

源码静态分析工具多种多样

源码静态分析工具以警告的形式告知开发人员源码中可能存在的缺陷

源码静态分析工具面临的主要问题—无效警告/误报



警告分类

- 警告：warning/alarm/alert/violation
- 无效警告（false positive/unactionable warnings）：开发人员没有进行修改的警告（i.e., the warnings that do not be acted on by developers.)
 1. 由于静态分析工具过估计程序行为产生的误报
 2. 涉及到过时代码产生的警告
 3. 不影响软件 and 用户使用的缺陷
 4. 不同开发者的编程技能
- 有效警告/正报（true positive/actionable warnings）：开发者动手修改的警告，（i.e., the warnings that are acted on by developers.)

警告数据集构建

有效警告识别技术

什么是有效警告识别？

有效警告识别 (actionable warning identification/postprocessing warnings)：

- 降低报告给开发者的警告数--Reducing the number of warnings by reporting them to users
- 利用额外的信息丰富警告进而降低人工审核的工作量--Reducing manual inspection effort by enriching warnings with additional information
- 在检测过程中提供帮助或工具支持以简化人工审核警告的过程--Simplifying manual inspection of warnings by providing assistance and tool support during the inspection process

参考文献：

☐ *Survey of Approaches for Postprocessing of Static Analysis Alarms*

有效警告识别分类

- 警告消减 Pruning of alarms
- 警告排序 Ranking of alarms
- 警告聚类 Clustering of alarms
- 无效警告自动过滤 Automated false positives elimination
- 基于静态和动态分析组合 Combination of static and dynamic analyses
- 人工审查简化 Simplification of manual inspection

警告消减

基于机器学习的警告消减

参考文献：

☐ *Finding Patterns in Static Analysis Alerts – Improving Actionable Alert Ranking*

☒ *Is There A “Golden” Feature Set for Static Warning Identification?*

☒ *Learning to Recognize Actionable Static Code Warnings (is Intrinsically Easy)*

☐ *Learning a Classifier for False Positive Error Reports Emitted by Static Code Analysis Tools*

☐ *Classifying False Positive Static Checker Alarms In Continuous Integration Using Convolutional Neural Networks*

☐ *An Empirical Assessment of Machine Learning Approaches for Triaging Reports of a Java Static Analysis Tool*

机器学习能够帮助挖掘有效警告和无效警告中模式差异，进而去分类有效警告和无效警告。然而，机器学习模型需要大量训练集。此外，哪一种类型的警告模式应该去挖掘以及如何呈现还需要进一步改进。

基于源代码演变的警告消减

通过源代码的演变来判断哪些警告是新增的，哪些警告是消失的，哪些警告是一直存在的？

该方法只适用项目源码有持续演变的情况，如果项目处于初始版本则该方法不适用。

警告排序

将更可能是有效警告的排在其他警告的前面

基于历史信息感知的警告排序

生存时间越短的警告越重要，因此应该分配越高的优先级，以便开发人员能够更快修复这些警告。

- 如何去识别修复的警告是很困难的
- 该方法要求项目具有大量的历史信息
- 跨项目的之间信息不能互用

基于用户反馈自适应的警告排序

参考文献：

- ☐ *Correlation Exploitation in Error Ranking*
- ☐ *Efindbugs- Effective error ranking for findbugs*

特点：

- 需要用户参与，用户标记警告的正确性？
- 依赖初始警告标记的有效性

基于多工具组合的警告排序

警告聚类

基于警告相似度或相关性对警告进行聚类

- Sound
- Unsound

特点：Sound 聚类只用审核每个类簇中的一个警告，该类簇 的其他警告则无需在审核，因此，该方法能够高效地提升审 核的速率，但是如何保证每个类簇的中 sound 时一个具有挑 战性的问题。Unsound 聚类主要基于警告的相似度对其进行 聚类，但是该方法可能会导致假阴性，即有效警告被遗漏。

无效警告自动过滤

基于模型检测或符号执行在没有用户介入的情况下精准删除无效警告

- 模型检测：模型检测的基本思想是用状态迁移系统(S)表示系统的行为，用模态 逻辑公式(F)描述系统的性质。这样“系统是否具有期望的性质”就转化为数学问 题“状态迁移系统 S 是否是公式 F 的一个模型”，用公式表示为 $S \models F$ 。对有穷状态系 统，这个问题是可判定的，即可以用计算机程序在有限 时间内自动确定。
- 符号执行：使用符号执行分析一个程序时，该程序会使用符号值作为输入，而 非一般执行程序时使用的具体值。在达到目标代码时，分析器可以得到相应的路 径约束，然后通过约束求解器来得到可 以触发目标代码的具体值。

特点：相较其他方法而言，该方法更加精确。但是，在大项目上，该 方法的扩展性较差且效率较低。

基于静态和动态分析组合

利用动态分析来验证静态分析报告的警告

特点：该方法验证后的警告具有较高的准确性，但是，该方法需要执行代码。

☐ *Validating Static Warnings via Testing Code Fragments*

人工审查简化

在人工审查警告的过程中简化其过程

- 半自动化警告审查
- 基于反馈的人工审查
- 基于检查表的人工审查
- 相关警告查询
- 警告自动修复建议
- 基于可视化的警告审查工具

特点：能为用户审核警告提供较大的帮助，但是也需要用户参与。

总结

方法类别	优点	缺点
警告消减	只有分类为有效的警告被检查，其他的被分类为无效的警告则不会报告给用户	由于被消减后的警告中不能保证是否还存在有效警告，因此该方法可能导致假阴性
警告排序	在审查过程中，将更可能是有效的警告排在列表的前面，这种方法不会导致假阴性	该方法一般需要审核所有的警告，用户反馈的方法需要人工介入
警告聚类	分组检查警告可以降低审查工作量	基于 unsound 聚类方法可能导致假阴性；基于 sound 聚类的方法审查工作量的降低依赖支配警告的数量
无效警告自动过滤	相比于其他方法，该方法更加自动化且更加精确	该方面在可扩展性和效率上还存在问题
基于动静态结合	能够展示有效警告的错误场景	需要实际执行程序
人工审查简化	在审查过程中能够提供较大的帮助	在审查过程中需要人工介入